Question 1: [2 marks]

(a) [1 mark]

Given: $\neg R$

From $(P \vee Q) \rightarrow R$ and $\neg R$, apply Modus Tollens:

$\neg R \rightarrow \neg(P \vee Q)$

$\neg(P \vee Q) = \neg P \wedge \neg Q$

Therefore, the door was not open and the motion sensor was not triggered.

(b) [1 mark]

From the universal rule: $\forall x\ (Cheat(x) \rightarrow Disqualified(x))$, instantiate for Alice:
$Cheat(Alice) \rightarrow Disqualified(Alice)$
Given: $\neg Disqualified(Alice)$
Apply Modus Tollens:
$\neg Disqualified(Alice) \rightarrow \neg Cheat(Alice)$
Conclusion: Alice did not cheat.

Question 2 [3 marks]

(a) [1 mark]
- Lower Bound: L_{Alert} = max(L_P, L_Q)
- Upper Bound: U_{Alert} = max(U_P, U_Q)

L_{Alert} = max(0.8, 0.3) = 0.8
U_{Alert} = max(0.9, 0.6) = 0.9
Since 0.7 = α < 0.8 < 0.9, the classification is: **A. Definitely true**

(b) [2 marks]

If the student's answer meets any of these, he will get full marks.

1. Expressing Uncertainty Explicitly

Bounds allow the system to represent how confident it is in a truth value.

2. Supporting Conservative Decision-Making

In safety-critical tasks (like autonomous driving), erring on the side of caution is crucial. If the lower bound is below the threshold, the system might wait or slow down rather than take action. This avoids making risky decisions based on overconfident estimates.

3. Robustness to Noisy or Incomplete Data

Sensors may fail or provide noisy signals. Bounds propagate uncertainty from inputs to outputs, allowing the system to track how uncertain the final decision is.

4. Better Interpretability

Engineers and operators can inspect bounds to understand how certain the model is about a decision. This improves debugging, transparency, and trust in the AI system.

Question 3 [2 marks]

(a) **TransE** is a translation-based embedding model that represents **entities and relations as vectors in the same space**. It assumes that for a valid triple (h, r, t), the embedding of the tail entity t should be **close to** the embedding of the head entity h plus the relation vector r, i.e.:

$$h + r \approx t$$

(b) **Scoring Function:**

TransE defines a distance-based score:

$$f(h, r, t) = || h + r - t ||_{L1\,/L2}$$

The **smaller** the score, the **more likely** the triple is true.

Question 4 [2 marks]

At each node of the decision tree, starting from the root, the algorithm assesses the potential impurity reduction for splitting the training data on each available feature (and for every possible split point for numeric features). *The best-performing split is selected without any look-ahead. That's the greedy part – there is no effort to craft an optimal tree overall, just a maximal local decision regarding the current split.* The algorithm is then recursively invoked on each resulting sub-tree.

[Note that answers that mentioned maximizing impurity reduction but did not mention the lack of looking ahead did not receive full marks. Generally, these responses received 1 mark out of 2, with ½ point variance depending on exactly what else was said.]

Question 5 [3 marks]

With classic logic both STRONG and HEAVY would be either True or False (by some criteria, like how much they can benchpress, or the athlete's weight in kg). *If, and only if, both criteria are true, then the individual is judged to be suitable for the sport of hammer throwing; else, not.*

With Fuzzy Logic, Fuzzy Set Theory can be applied to both STRONG and HEAVY. So some level of strength performance will map to μ_s, and some bodyweight to μ_h. Both membership functions are in range [0,1] and take any value in that range. The AND function might be implemented as $\min(\mu_s, \mu_h)$ [there are other options, such as the Yager operator, or multiplying the two memberships]. The THEN might have strength 1.0, or something less as a further multiplier. *The final result is a suitability score for hammer throwing anywhere in the range [0,1].*

Question 6 [3 marks]

The key here is understanding that a fitness function in a GA dictates the suitability of an individual to pass their genome onto the next generation.

Elements of the fitness function for this case should include:
Difference of target speed to actual speed
Difference of target direction to actual direction
Difference of target height to actual height
Also, possibly, maintaining pitch, yaw and roll within acceptable bounds

The highest fitness score would be attained where all the above differences are low across a simulation run (reinforcement learning). So, the fitness function might put a minus sign in front of absolute value (or square) of each summed over observations during the simulation run.